

SECTION 26 — PERFORMANCE / RELIABILITY AWARENESS

Purpose: Low-level performance engineering mechanics, k6/JMeter scripting patterns, latency distributions (p95/p99), reliability testing, and suite execution optimization for Enterprise Senior SDET / Staff QA Architect assessment (Deloitte, Big 4, FAANG).

26.1 Performance Testing Fundamentals

Low-Level Execution Loops & Load Metrics

Performance testing evaluates system behavior under varying traffic profiles to verify compliance with Service Level Objectives (SLOs) and Service Level Agreements (SLAs).

- **Response Time:** Time elapsed from client request dispatch to full response byte consumption.
- **Latency vs Processing Time:** Latency is network transit delay; processing time is server-side computation + DB wait time.
- **Throughput (RPS/TPS):** Requests/Transactions Per Second processed by the system.
- **Concurrency vs Virtual Users (VUs):** Concurrency is the number of requests actively executing on server threads at a microsecond t ; Virtual Users are client threads executing user scenarios (which include think-time and pacing).

LATENCY & THROUGHPUT DISTRIBUTION	
Percentile	Target Threshold (SL0)
p50 (Median)	< 150 ms
p90	< 300 ms
p95 (Standard SLA)	< 450 ms
p99 (Tail Latency)	< 800 ms
Error Rate	< 0.01% (99.99% Reliability)

26.2 Throughput, Concurrency, and Little's Law

Little's Law Formula in Performance Engineering

Little's Law dictates the relationship between Concurrent Users (N), Throughput (X), and Average Response Time (R) plus Think Time (Z):

$$N = X \times (R + Z)$$

- If target Throughput $X = 500 \text{ RPS}$, Average Response Time $R = 0.2 \text{ s}$, and Think Time $Z = 1.8 \text{ s}$:
 $N = 500 \times (0.2 + 1.8) = 1,000 \text{ Virtual Users}$

26.3 Load Profiles & Testing Strategies

Load Profile	Traffic Pattern	Primary Objective
Ramp-up / Baseline	Linear increase to expected peak	Verify p95/p99 under normal SLA
Stress Testing	Ramp past peak until system breaks	Identify breakpoint & failure mode
Spike Testing	Instant 5x-10x traffic burst	Verify autoscale & recovery time
Soak / Endurance	Constant peak load for 24-72 hours	Detect memory leaks & connection pool leaks

26.4 Production-Ready k6 Load Testing Script (TypeScript)

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Options } from 'k6/options';

// 1. Load Profile Options Configuration
export const options: Options = {
  stages: [
    { duration: '2m', target: 100 }, // Ramp-up to 100 VUs
    { duration: '5m', target: 100 }, // Stay at 100 VUs (Steady Load)
    { duration: '2m', target: 300 }, // Ramp-up to 300 VUs (Stress Level)
    { duration: '5m', target: 300 }, // Stay at 300 VUs
    { duration: '2m', target: 0 }, // Ramp-down to 0 VUs
  ],
  thresholds: {
    http_req_failed: ['rate<0.01'], // Error rate < 1%
    http_req_duration: ['p(95)<450', 'p(99)<800'], // p95 < 450ms, p99 < 800ms
  },
};

// 2. Scenario Execution Function
export default function () {
  const url = 'https://api.staging.bank.com/v1/accounts/transfer';
  const payload = JSON.stringify({
    fromAccount: 'ACC-100293',
    toAccount: 'ACC-992014',
    amount: 150.00,
    currency: 'USD',
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${__ENV.TEST_AUTH_TOKEN}`,
    },
  };
};
```

```

const res = http.post(url, payload, params);

// 3. Strict Verification & Assertions
check(res, {
  'status is 200 or 201': (r) => r.status === 200 || r.status === 201,
  'transaction completed within 500ms': (r) => r.timings.duration < 500,
  'has transaction reference': (r) => (r.json('transactionId') as string).length > 0,
});

// Pacing / Think Time: Simulate realistic user delay
sleep(Math.random() * 2 + 1); // Random sleep 1s-3s
}

```

26.5 High-Stakes Triage Scenario: Tail Latency Explosion (p99 Spike)

Scenario Setup

During a 1,000 VU load test on an order processing API, **p50** latency remains healthy at 120ms, but **p99** latency spikes to 14.5 seconds with intermittent **HTTP 504 Gateway Timeout** errors.

Senior Diagnostic Playbook

- 1. Thread & Connection Pool Analysis:** Inspect HikariCP connection pool metrics in Datadog/CloudWatch. If **ActiveConnections == MaxPoolSize** while **WaitThreads > 0**, backend threads are blocking waiting for DB connections.
- 2. Garbage Collection Stop-The-World Pauses:** Check JVM GC logs for G1GC / ZGC long pauses (`$> 2\text{s}$`). Unoptimized heap settings (**-Xms** set lower than **-Xmx**) trigger frequent full GC pauses during memory allocation spikes.
- 3. Database Lock Contention:** Check PostgreSQL **pg_stat_activity** for query lock contention (**WAIT_EVENT_TYPE == 'Lock'**). High concurrency on a single row (e.g., updating inventory count without optimistic locking) forces serial execution.

```

-- Query to identify blocking queries in PostgreSQL during perf run
SELECT
  blocked_locks.pid      AS blocked_pid,
  blocked_activity.username AS blocked_user,
  blocking_locks.pid     AS blocking_pid,
  blocking_activity.username AS blocking_user,
  blocked_activity.query  AS blocked_statement
FROM   pg_catalog.pg_locks      blocked_locks
JOIN   pg_catalog.pg_stat_activity blocked_activity ON blocked_activity.pid = blocked_locks.pid
JOIN   pg_catalog.pg_locks      blocking_locks
      ON blocking_locks.locktype = blocked_locks.locktype
      AND blocking_locks.database IS NOT DISTINCT FROM blocked_locks.database
      AND blocking_locks.relation IS NOT DISTINCT FROM blocked_locks.relation
      AND blocking_locks.page IS NOT DISTINCT FROM blocked_locks.page
      AND blocking_locks.tuple IS NOT DISTINCT FROM blocked_locks.tuple
      AND blocking_locks.virtualxid IS NOT DISTINCT FROM blocked_locks.virtualxid
      AND blocking_locks.transactionid IS NOT DISTINCT FROM blocked_locks.transactionid
      AND blocking_locks.classid IS NOT DISTINCT FROM blocked_locks.classid

```

```

AND blocking_locks.objid IS NOT DISTINCT FROM blocked_locks.objid
AND blocking_locks.objsubid IS NOT DISTINCT FROM blocked_locks.objsubid
AND blocking_locks.pid != blocked_locks.pid
JOIN pg_catalog.pg_stat_activity blocking_activity ON blocking_activity.pid = blocking_locks.pid
WHERE NOT blocked_locks.granted;

```

26.6 Suite Execution Optimization (SDET Automation Performance)

Optimization Strategies for E2E Suites

1. **Bypass UI Authentication via StorageState:** Reusing cached `auth.json` saves 12-15 seconds per test case.
2. **Resource Abort & Interception:** Abort non-essential static assets (images, fonts, Google Analytics scripts) during E2E runs:

```

// Playwright Route Interception for Fast Execution
await page.route('**/*.{png,jpg,jpeg,svg,gif,woff,woff2}', route => route.abort());
await page.route('**/google-analytics.com/**', route => route.abort());

```

3. **API-Based Precondition Provisioning:** Replace UI form setup with parallel API calls (`APIRequestContext`) before launching browser contexts.

26.7 Anti-Patterns in Performance Engineering

- **Averaging Latency Values:** Relying on arithmetic mean latency masks critical tail latency spikes (\$p99\$).
- **Single-User Data Reuse:** Sharing a single login/user account across 500 VUs causes unrealistic DB cache hits and session eviction collisions.
- **Ignoring Think Time & Pacing:** Running VUs in an infinite zero-delay loop overwhelms API gateways unrealistically and generates false client-side bottlenecks.
- **Testing Without Production-Scale Data Volume:** Running load tests against an empty database table masks missing index performance collapses.